

《单片机原理及应用》

复习



考试提纲

ILO 序号 ^o	ILO-1 ^o	ILO-2 ^o	ILO-3 ^o	ILO-4 ^o	ILO-5 ^o
预期学习结果 ^o	单片机结构原理的基本认知: 理解单片机的总体结构, CPU、程序存储器、数据存储器、并行 I/O 口结构; 理解单片机的时钟系统与复位系统, 设计单片机时钟电路与复位电路; 理解单片机的电源管理方式。 ^o	单片机汇编指令与单片机 C 语言: 了解单片机汇编指令系统; 理解单片机 C 语言与标准 C 语言的区别; 应用单片机 C 语言实现单片机程序的设计。 ^o	单片机人机交互接口电路: 理解单片机基本按键接口电路, 应用 C 语言实现按键驱动程序; 理解单片机基本的 LED 与数码管显示接口电路, 应用 C 语言实现显示驱动程序。 ^o	单片机定时/计数器: 理解单片机定时/计数器的结构和原理; 理解单片机定时/计数器的工作方式; 应用单片机定时/计数器实现具体应用。 ^o	单片机中断系统: 理解单片机中断系统结构与原理、中断响应的处理过程; 理解单片机中断源, 中断程序结构与中断系统应用。 ^o
ILO 序号 ^o	ILO-6 ^o	ILO-7 ^o	ILO-8 ^o	ILO-9 ^o	^o
预期学习结果 ^o	单片机数模转换与模数转换: 理解单片机 A/D 转换器, 单片机 D/A 转换器结构与原理; 应用单片机 A/D 转换器、D/A 转换器实现数据采集和	单片机串行通信接口: 理解串行通信基本概念; 理解单片机串行口的结构和工作方式; 应用单片机串行口实现通	应用系统模块设计与实现: 课堂讨论和实验项目中能够根据要求进行单片机应用系统硬件单元电路设计与实现, 能	仿真软件应用: 基本掌握单片机软件开发仿真平台 Keil μ vision 的使用, 应该该软件进行单片机应用程序设计与仿	^o

一、二、三、



第一部分

1. 存储器结构
2. 中断
3. I/O
4. 复位与时钟
5. 按键与显示

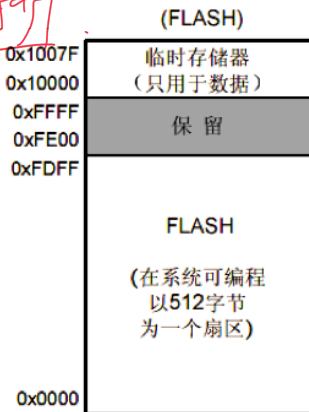
第二部分

1. 定时器
2. 串口
3. AD/DA



C8051F020存储器结构

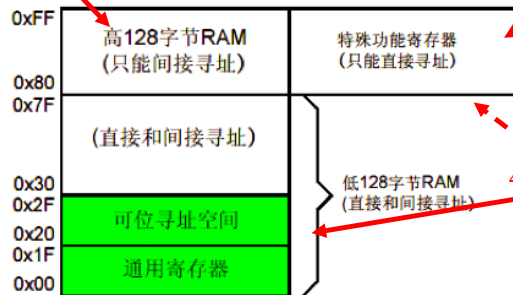
1. 程序存储器



1. 程序存储器

2. 内部数据存储器

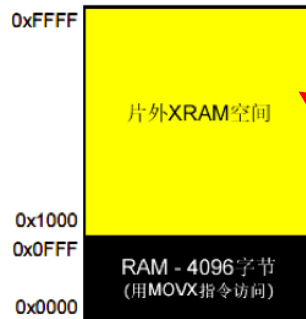
内部数据地址空间



3. 特殊功能寄存器

4. 位寻址区

外部数据地址空间



5. 外部数据存储器



- CIP-51有哪些存储空间？各个存储空间的功能及寻址范围是什么？

答：有程序存储器、内部数据存储器、外部数据存储器、特殊功能寄存器和位寻址区五个独立的地址空间。

程序存储器：存放程序和常数，容量为64KB，地址范围0000H-0FFFFH；

内部数据存储器：用于通用寄存器和存放数据的临时存储器，容量为256B，地址范围00H-0FFH；

外部数据存储器：用于存放数据，容量为64KB（有4KB在芯片内部），地址为0000-0FFFFH；

特殊功能寄存器（SFR）：提供对片内资源和外设的访问和控制，以及与外设的数据交换，C8051F020有122个，离散分布在地址80H-0FFH中；

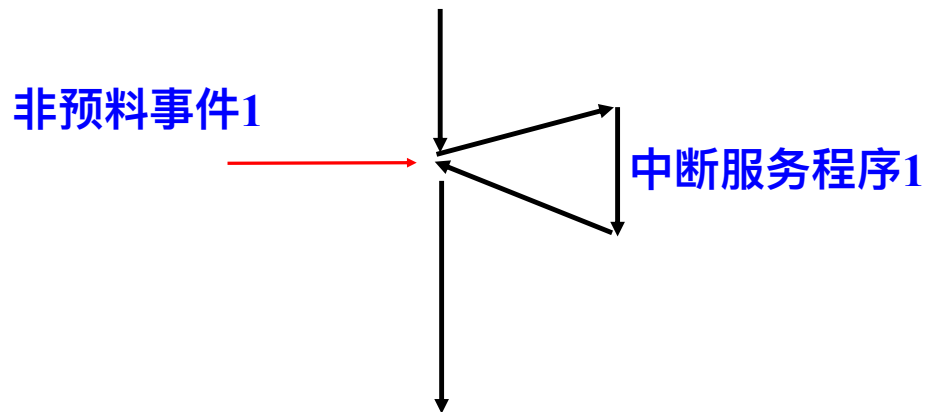
位地址空间：存放位变量，进行布尔运算，容量为256位，有内部RAM的20H-2FH（16字节，128位）和部分SFR（字节地址可以被8整除的）组成，地址范围为00H-0FFH



2.3 中断系统

• 2.3.1 中断概念

CPU执行流程



中断的定义：在计算机运行过程中，发生紧急请求（如按键盘），CPU暂停当前工作，转而处理紧急请求，处理完毕后再回到原来被中断的地方，继续运行原来的工作。



中断执行过程

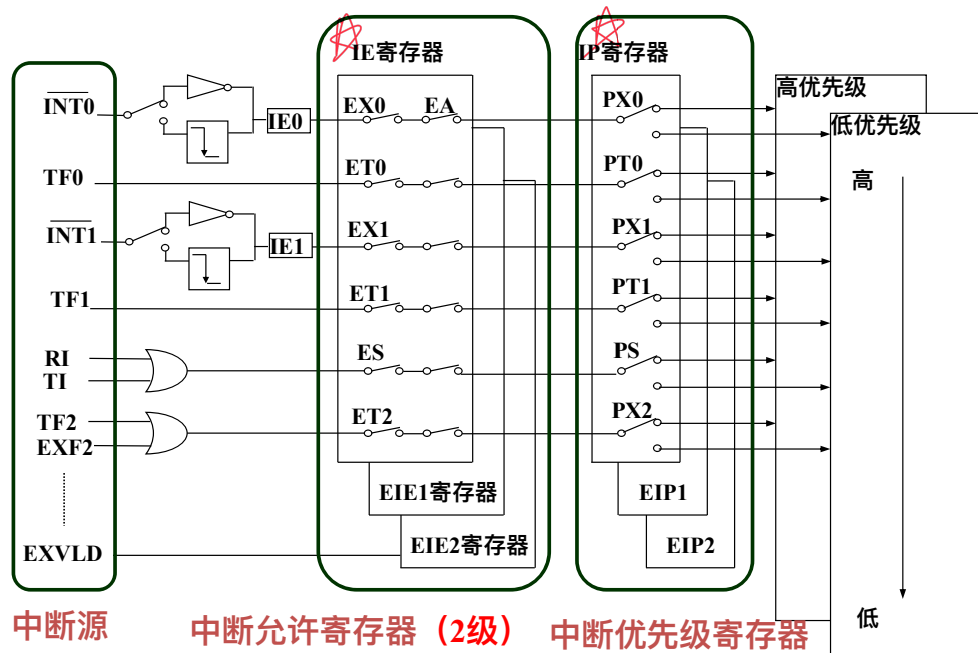
中断允许. 中断 ↓



- ▶ 当一个外设或者外部源满足有效的中断条件时，相应的**中断标志被置为逻辑“1”**
- ▶ 如果中断被允许，在中断标志**被置位**时将产生中断，一旦**当前指令执行完，开始执行中断服务程序（ISR）**
- ▶ 每个ISR以RETI指令结束，使程序**回到中断前执行完的那条指令的下一条指令**



C8051F中断系统





CPU响应中断后，由硬件自动执行如下的功能操作：

- (1) 根据中断请求源的优先级高低，对相应的优先级状态触发器置1（不可寻址，该触发器指出CPU开始处理的中断优先级）。
- (2) 保护断点，即把程序计数器PC的内容压入堆栈保存。
- (3) 清内部硬件可清除的中断请求标志位(IE0、IE1、TF0、TF1；有些中断源如RI、TI不能被清除，需要中断服务程序软件清除)。
- (4) 把被响应的中断服务程序入口地址（中断向量）送入PC，从而转入相应的中断服务程序执行。各中断服务程序的入口地址见表。



 寧波大學
NINGBO UNIVERSITY

表 2-5 C8051F020 单片机的中断源

ADC0 窗口比较	0x0043	中 断 源	中断向量	优先级	中 断 标 志	使 能	优先级控制
可编程计数器阵列	0x004B	定时器 4 溢出	0x0083	16	TF4(T4CON, 7)	ET4(EIE2, 2)	PT4(EIP2, 2)
		ADC1 转换结束	0x008B	17	AD1INT(ADC1CN, 5)	EADC1(EIE2, 3)	EADC1(EIE2, 3)
		外部中断 6	0x0093	18	IE6(PRT3IF, 5)	EX6(EIE2, 4)	PX6(EIP2, 4)
比较器 0 下降沿	0x0053	外部中断 7	0x009B	19	IE7(PRT3IF, 6)	EX7(EIE2, 5)	PX7(EIP2, 5)
比较器 0 上升沿	0x005B	UART1	0x00A3	20	RI(SC0N1, 0)	ES1(EIE2, 6)	PS1(EIP2, 6)
比较器 1 下降沿	0x0063				TI(SC0N1, 1)		
比较器 1 上升沿	0x006B	外部晶体振荡器准备好	0x00AB	21	XTLVLD(OSCXNCN, 7)	EXVLD(EIE2, 7)	PXVLD(EIP2, 7)
定时器 3 溢出	0x0073						
ADC0 转换结束	0x007B						

1. 中断源

(1) 外部中断源

- 与**MCS-51兼容的两个外部中断源**，外部中断0 (/INT0) 和外部中断1 (/INT1) 。
 - ▶ 可配置为低电平触发或下降沿触发。由IT0 (TCON.0) 和IT1 (TCON.2) 的设置决定。
 - ▶ 如果被配置为**边沿触发**，CPU转向ISR时自动清除相应的中断标志
 - ▶ 如果被配置为**低电平触发**，中断标志将跟随外部中断输入引脚的状态



外部中断0 (/INT0) 和外部中断1 (/INT1) 的使用

(1) $EA = 1, EX0 = 1$ 或者 $EX1 = 1$

(2) 触发方式：电平触发或边沿触发

IT0 (TCON[0]) 或 IT1 (TCON[2])：置为0，电平触发；置为1，边沿触发。

	D7	D6	D5	D4	D3	D2	D1	D0	字节地址
TCON	TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0	98H

(3) 引脚配置

XBR1[2]或XBR1[4]置为1

R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	复位值
SYSCKE	T2EXE	T2E	INT1E	T1E	INT0E	T0E	CP1E	00000000
位7	位6	位5	位4	位3	位2	位1	位0	SFR地址:

2. 中断控制

(1) 中断使能控制

- C8051F对所有的中断源的开放和屏蔽（总控制位EA）
- 每个中断源都有中断允许控制
- 受中断允许寄存器IE、EIE1、EIE2控制。

(2) 中断优先级别的设定

- 每个中断源都可以**设置为高优先级和低优先级**，由中断优先级寄存器IP、EIP1、EIP2统一管理。



中断允许寄存器IE

位7	位6	位5	位4	位3	位2	位1	位0	复位值
EA	IEGF0	ET2	ES0	ET1	EX1	ET0	EX0	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xA8

位7: EA位, EA=0: 禁止所有中断, EA=1: CPU开放中断, 每个中断位是开放还是屏蔽由各种的允许位确定

位6: IEGF0, 通用标志0。用作软件控制的通用标志位

位5: ET2, 定时器2中断允许位

位4: ES0, 串行口UART0中断允许位

位3: ET1, 定时器1中断允许位

位2: EX1, 外部中断1 (/INT1) 中断允许位

位1: ET0, 定时器0中断允许位

位0: EX0, 外部中断0 (/INT0) 中断允许位



• 中断允许寄存器 EIE1

位7	位6	位5	位4	位3	位2	位1	位0	复位值
ECP1R	ECP1F	ECP0R	ECP0F	EPCA0	EWADC 0	ESMB 0	ESP10	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xE6

位7: ECP1R, 允许比较器1 (CP1) 上升沿中断

位6: ECP1F, 允许比较器1 (CP1) 下降沿中断

位5: ECP0R, 允许比较器0 (CP0) 上升沿中断

位4: ECP0F, 允许比较器0 (CP0) 下降沿中断

位3: EPCA0, 允许可编程计数器阵列 (PCA0) 中断

位2: EWADC0, 允许ADC0窗口比较中断

位1: ESMB0, 允许SMBus0中断

位0: ESP10, 允许串行外设接口0中断



• 中断允许寄存器 EIE2

位7	位6	位5	位4	位3	位2	位1	位0	复位值
EXVLD	ES1	EX7	EX6	EADC1	ET4	EADC0	ET3	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xE7

位7: EXVLD, 允许外部时钟源有效 (XTLVLD) 中断

位6: ES1, 允许UART1中断

位5: EX7, 允许外部中断7

位4: EX6, 允许外部中断6

位3: EADC1, 允许ADC1转换结束中断

位2: ET4, 允许定时器4中断

位1: EADC0, 允许ADC0转换结束中断

位0: ET3, 允许定时器3中断



 宁波大学
NINGBO UNIVERSITY



优先级管理

- 中断优先级只有高低两级。若有两个或两个以上中断源处于同一中断优先级，内部中断系统对各中断源的处理遵循以下两条基本原则：
 - 低优先级中断可以被高优先级中断所中断，反之不能。
 - 一种中断（不管是什么优先级）一旦得到响应，与它同级的中断不能再中断它。
- 由两个不可寻址的“优先级激活”触发器实现上述基本原则。
- 同时收到几个同一优先级的中断请求时，哪一个请求将得到服务，取决于内部的硬件查询次序，CPU将按自然优先级顺序确定该响应哪个中断请求。
多位中断最高



自然优先级由硬件形成

中断源

外部中断0

定时器0中断

外部中断1

定时器1中断

串行口0中断

定时器2中断

.

.

.

串行口1中断

外部晶振准备好

同级自然优先级

最高级



最低级

考题：自然优先级最高的中断源是 **复位**



中断响应时间

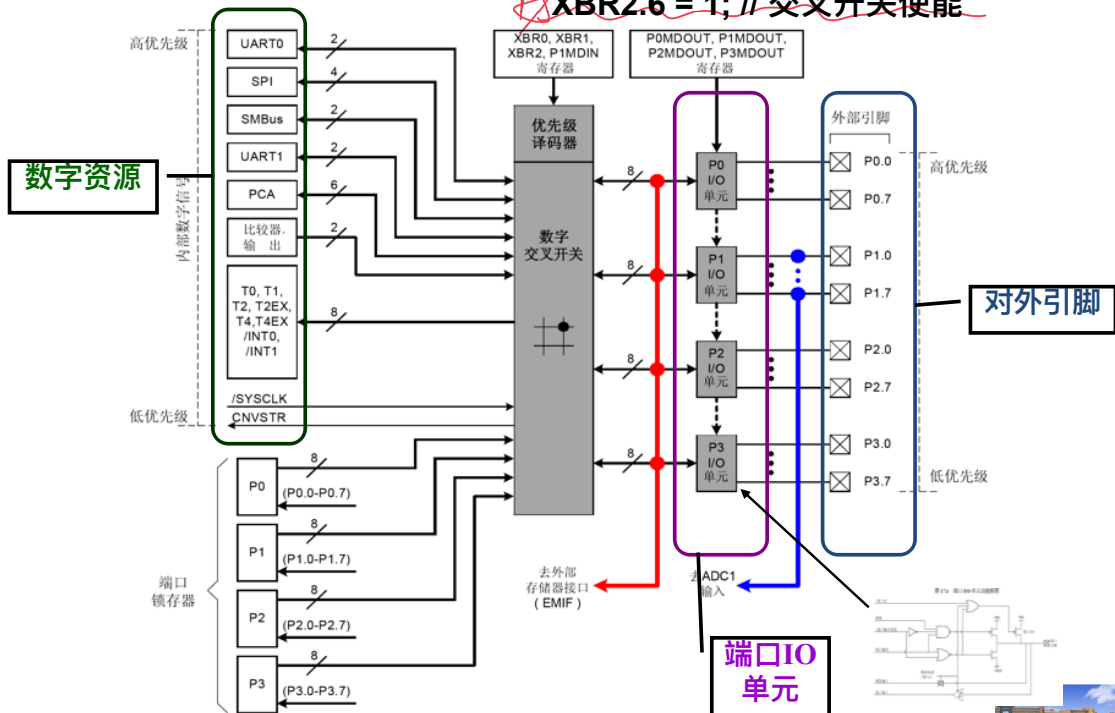
- ▶ **最快**的响应时间为**5个**系统时钟周期：一个周期用于检测中断，4个周期完成对ISR的长调用（LCALL）。
- ▶ **最长**的响应时间是下一条指令是**DIV**的情况。响应时间为**18**个系统时钟周期：
 - 1个时钟周期检测中断；
 - 5个时钟周期执行RETI；
 - 8个时钟周期完成DIV指令；
 - 4个时钟周期执行对ISR的长调用（LCALL）



交叉开关设置

配置XBR0、XBR1、XBR2;

XBR2.6 = 1; // 交叉开关使能



优先级交叉开关译码表

请解点心用

高

UART0

SPI

SMBus

UART1

PCA

	P0								P1								P2								P3								交叉开关寄存器位	
引脚 I/O	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7		
TX0	●																																UART0EN:XBR0.2	
RX0		●																																
SCK	●		●																														SPI0EN:XBR0.1	
MISO		●		●																														
MOSI			●		●																													
NSS				●		●																												
SDA	●		●			●																											SMB0EN:XBR0.0	
SCL		●		●			●																											
TX1	●		●			●																											UART1EN:XBR2.2	
RX1		●		●			●						●																					
CEX0	●		●			●							●																					
CEX1		●		●			●						●																					
CEX2			●		●			●					●																					
CEX3				●		●			●					●																				
CEX4					●		●			●					●																			
ECI	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●																	ECI0E:XBR0.6	
CP0	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●																CP0E:XBR0.7	
CP1	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●															CP1E:XBR1.0	
T0	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●														T0E:XBR1.1	
/INT0	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●													INT0E:XBR1.2	
T1	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●												T1E:XBR1.3	
/INT1	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●											INT1E:XBR1.4	
T2	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●											T2E:XBR1.5	
T2EX	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●										T2EXE:XBR1.6	
T4	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●									T4E:XBR2.3	
T4EX	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●							T4EXE:XBR2.4	
/SYSCLK	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●						SYSCKE:XBR1.7
CNVSTR	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●					CNVSTE:XBR2.0

低

交叉开关寄存器被正确配置后
通过将XBR1 (XBR2.6)
设置为逻辑'1'来使能交叉开关。



I/O引脚初始化—输出：推挽还是开漏；输入：模拟还是数字

1. 数字输出

- 设置输出方式为“推挽”

例如：

设置P3MDOUT.7 = 1，即可将P3.7 配置为

数字推挽输出。（P3MDOUT = 0x80）*寄存器赋值。*

例如：

设置P3MDOUT.7= 0，且P3.7 =1，即

可将P3.7 配置为数字输入。

例如：

设置P1MDOUT.7= 0，且P1.7 =1，且

P1MDIN.7 = 0，即可将P1.7 配置为模

拟输入。

2. 数字输入（计数器）

- 设置输出方式为“漏极开路”，逻辑0
- 向端口数据寄存器中的相应位写‘1’

3. 模拟输入（ADC）

- 设置输出方式为“漏极开路”，逻辑0
- 向端口数据寄存器中的相应位写‘1’
- 向P1MDIN 寄存器中的对应位写‘0’



2.6 复位与时钟

• 2.6.1 复位源

1、复位状态

复位电路可以将控制器置于一个预定的状态——**复位状态**。

在进入**复位状态**时，将发生以下过程：

- ▶ CIP-51 CPU停止程序执行
- ▶ 特殊功能寄存器（SFR）被初始化为所定义的复位值
- ▶ 外部端口引脚被置于一个已知状态
- ▶ 中断和定时器被禁止

端口I/O锁存器的复位值为0xFF（全部为逻辑‘1’），

外部I/O引脚处于高电平状态。复位之后弱上拉被使能。



C8051F020的7个复位源：

- 上电/掉电复位
- 外部/RST引脚复位
- 外部CNVSTR信号复位
- 软件命令复位
- 比较器0复位
- 时钟丢失检测器
- 看门狗定时器超时复位

20年考题：7个复位源的含义是？

- ✓ 上电/掉电复位：当电源电压超过预定值或低于预定阈值时使单片机复位。
- ✓ 外部/RST引脚复位：当复位引脚为低电平时使单片机复位。
- ✓ 外部CNVSTR信号复位：当外部功能引脚CNVSTR为低电平时复位单片机。
- ✓ 软件命令复位：当给单片机特殊功能寄存器的控制位写入1时，使单片机复位。
- ✓ 比较器0复位：当单片机内比较器0，如外部给定电压比较后为低电平时，使单片机复位。
- ✓ 时钟丢失检测器：当单片机时钟监测电路检测到单片机系统时钟丢失时将复位单片机。
- ✓ 看门狗定时器超时复位：当看门狗定时器溢出时，将复位单片机。



禁止 WDT：

向 WDTCN 寄存器写入 0xDE 后再写入 0xAD 将禁止 WDT。

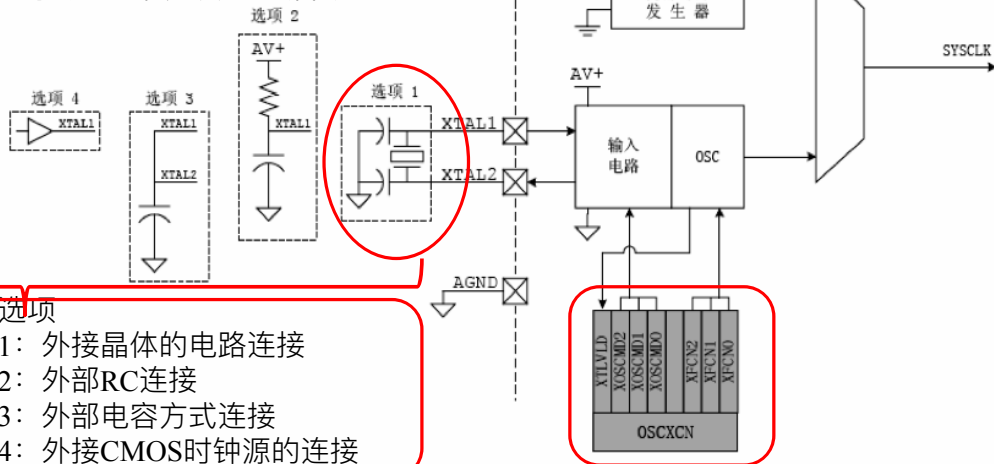
```
EA = 0;           //禁止所有中断
WDTCN = 0xDE;      //禁止软件看门狗定时器
WDTCN = 0xAD;
EA = 1;           //重新允许中断
```

写 0xDE 和写 0xAD 必须发生在 4 个时钟周期之内，否则禁止操作将被忽略。在这个过程期间应禁止中断，以避免两次写操作之间有延时。



2.6.5 C8051F单片机的时钟系统

- 一个内部振荡器和一个外部振荡器电路，可自由切换；分别由OSCICN、OSCXCN两个SFR控制
- 选择内部振荡器时，外部振荡器可保持在允许状态，可避免系统时钟切换时的启动延时



当外部晶体振荡器稳定运行时，晶体振荡器有效标志（OSCXCN 寄存器中的XTLVLD）被硬件置‘1’。

XTLVLD 检测电路要求在使能振荡器工作和检测XTLVLD 之间至少有**1 ms**的启动时间。（在外部振荡器稳定之前就切换到外部振荡器可能导致不可预见的后果。）

切换外部晶振 建议的过程为：

1. 使能外部振荡器
2. 等待至少1ms
3. 查询XTLVLD == '1'
4. 将系统时钟切换到外部振荡器

//外部24MHz晶振初始化程序

```
void Oscillator_Init()
```

```
{
```

```
    int i = 0;
```

```
    OSCXCN = 0x67; //若0x77,晶振2分
```

频

```
    for (i = 0; i < 3000; i++); //等1ms以上
```

```
    while ((OSCXCN & 0x80) == 0); //判断
```

XTLVLD是否为1

```
    OSCICN = 0x0C; //CLKSL置位
```

```
}
```

切换到外部时钟.



寻址方式 (1)

- 8051单片机的寻址方式有七种： $\rightarrow$ 七种。

- 1. 寄存器寻址

指令中地址码给出的是某一通用寄存器的编号，寄存器的内容为操作数。

MOV A, R0 ; $A \leftarrow (R0)$

可用寄存器寻址的空间是：R0~R7, ACC, DPTR, B

- 2. 直接寻址

直接寻址时，指令中地址码部分直接给出操作数的有效地址。

MOV A, 4FH ; $A \leftarrow (0x4F)$

可用直接寻址的空间是：内部RAM的低128B（包括其中可位寻址的区域）、特殊功能寄存器

- 3. 寄存器间接寻址

指令中给出的寄存器的内容为操作数的地址，而不是操作数本身，即寄存器为地址指针。R0、R1、DPTR, PC。

MOV A, @R1 ; $A \leftarrow (R1)$

可用R0或R1间接寻址片内或片外RAM的256B范围，可用DPTR或PC间接寻址64KB外部RAM或ROM



寻址方式(2)

- 4. 立即寻址

指令中地址码部分给出的就是操作数。

MOV A, #8FH ; A ← 0x8F

- 5. 变址寻址

变址寄存器的内容与偏移量相加，结果作为操作数的地址。

MOVC A, @A+DPTR ; A ← (A) + (DPTR)

只适用于程序存储器，通常用于读取数据表

- 6. 相对寻址

由PC提供的基地址与偏移量rel相加，得到操作数的地址。

SJMP rel ; PC ← (PC) + 2 + rel

- 7. 位寻址

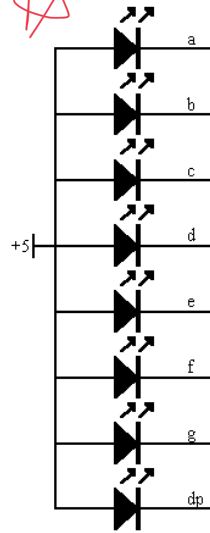
位寻址时，操作数是二进制数的某一位。

SETB bit ; (bit) ← 1

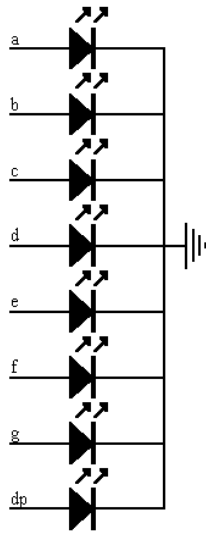
内部RAM的可位寻址区和SFR区中字节地址可以被8整除的寄存器所占空间。



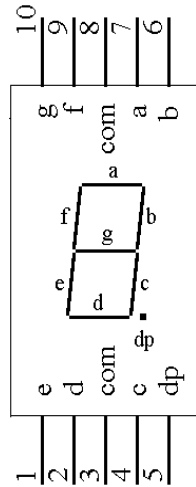
(1) LED数码管显示器结构



共阳极



共阴极



外形图



7段LED字型码 (共阳极)

数据位	D7	D6	D5	D4	D3	D2	D1	D0	共阳极编码	
字段	dp	g	f	e	d	c	b	a	不带小数点	带小数点
0	1/0	1	0	0	0	0	0	0	0xC0	0x40
1	1/0	1	1	1	1	0	0	1	0xF9	0x79
2	1/0	0	1	0	0	1	0	0	0xA4	0x24
3	1/0	0	1	1	0	0	0	0	0xB0	0x30
4	1/0	0	0	1	1	0	0	1	0x99	0x19
5	1/0	0	0	1	0	0	1	0	0x92	0x12
6	1/0	0	0	0	0	0	1	0	0x82	0x02
7	1/0	1	1	1	1	0	0	0	0xF8	0x78
8	1/0	0	0	0	0	0	0	0	0x80	0x00
9	1/0	0	0	1	0	0	0	0	0x90	0x10
全灭	1/0	1	1	1	1	1	1	1	0xFF	0x4F



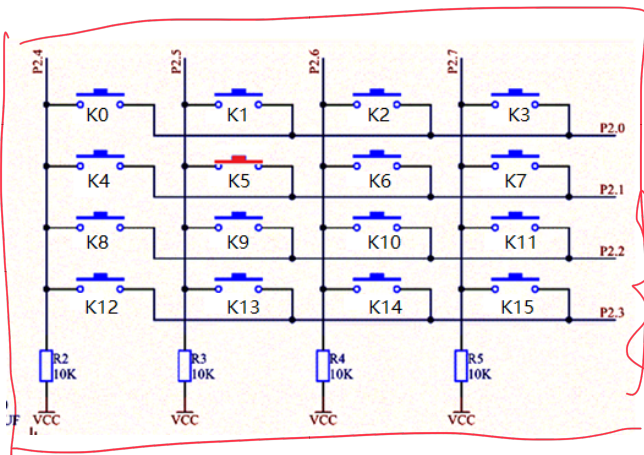
7段LED字型码 (共阴极)

数据位	D7	D6	D5	D4	D3	D2	D1	D0	共阴极编码	
字段	dp	g	f	e	d	c	b	a	不带小数点	带小数点
0	0/1	0	1	1	1	1	1	1	0x3F	0xBF
1	0/1	0	0	0	0	1	1	0	0x06	0x86
2	0/1	1	0	1	1	0	1	1	0x5B	0xDB
3	0/1	1	0	0	1	1	1	1	0x4F	0xCF
4	0/1	1	1	0	0	1	1	0	0x66	0xE6
5	0/1	1	1	0	1	1	0	1	0x6D	0xED
6	0/1	1	1	1	1	1	0	1	0x7D	0xFD
7	0/1	0	0	0	0	1	1	1	0x07	0x87
8	0/1	1	1	1	1	1	1	1	0x7F	0xFF
9	0/1	1	1	0	1	1	1	1	0x6F	0xEF
全灭	0/1	0	0	0	0	0	0	0	0x00	0x80



矩阵连接键盘

工作原理：键按矩阵排列,各键处于矩阵行/列的结点处,CPU通过对连在**行(列)**的I/O线送已知电平的信号,然后读取**列(行)**线的状态信息。逐线扫描,得出键值。



矩阵式键盘中,行、列线分别连接到按键开关的两端,列线通过上拉电阻接到+5V上。

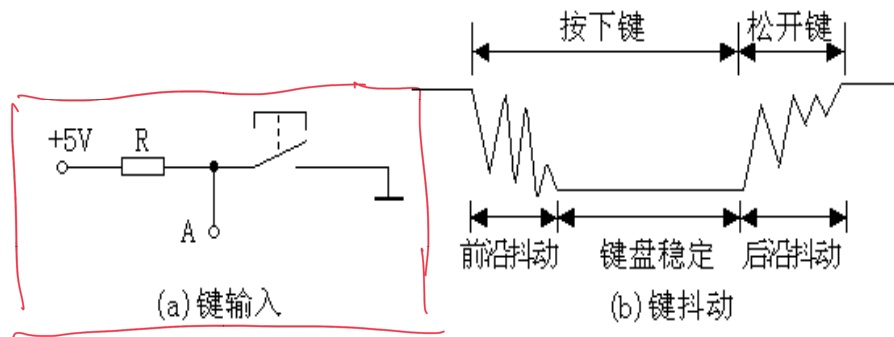
当无键按下时,列线处于高电平状态;

当有键按下时,行、列线将导通,此时,列线电平将由与此列线相连的行线电平决定。



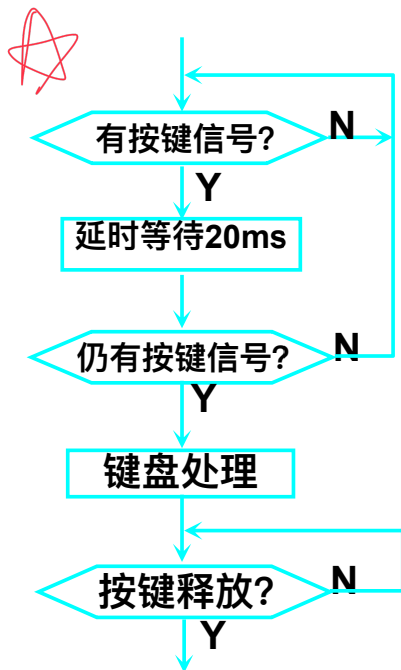
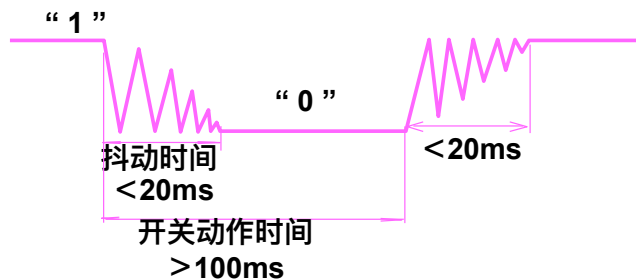
键盘驱动程序所需要解决的三个问题

- (1) 键抖动及处理
- (2) 键盘状态的判断—是否有键按下
- (3) 闭合键的识别—是哪个键按下了



键抖动及处理

(1) 软件延时去抖



键盘扫描程序的编写

(1) 判别有无键按下。

(2) 延时。

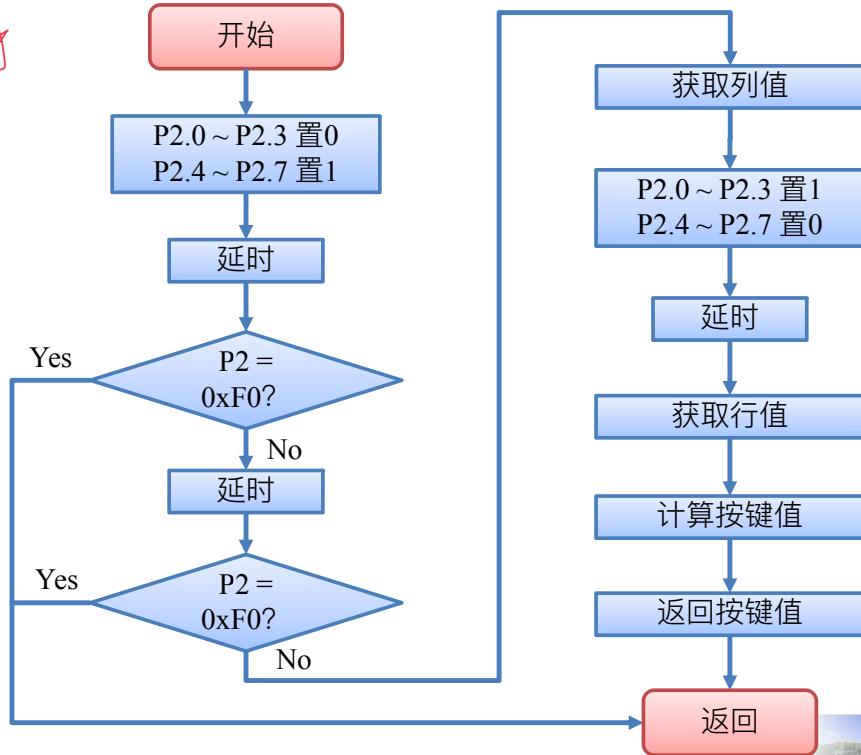
(3) 键盘扫描取得闭合键的行、列值—键值。

(4) 用算法或查表法得到键号。

***(5) 判断闭合键是否释放，如没释放则继续等待。**

(6) 将闭合键键号保存, 然后返回。



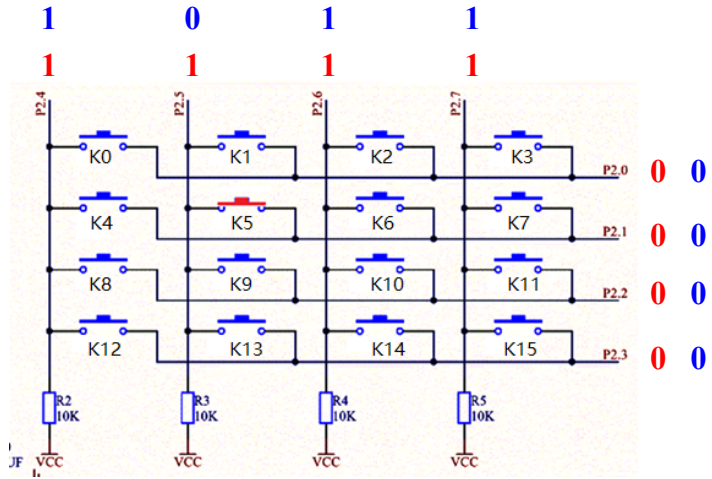


扫描列值

P2 = 0xF0;

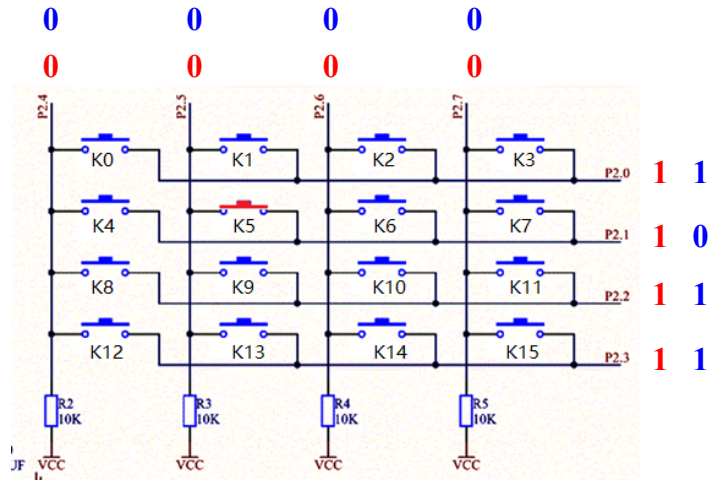
```
temp1 = P2;
```

//此时temp1为0xD0



扫描行值

```
P2 = 0x0F;
temp2 = P2;           //此时temp2为0x0D
```



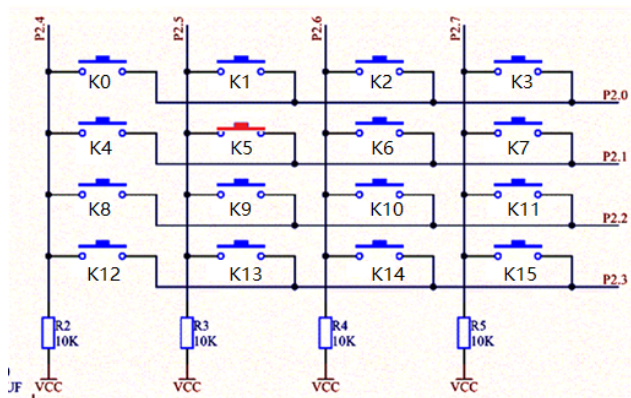
计算键值

```
temp1 |= temp2;           //temp1 = 0xDD
```

temp1: 1 1 0 1 0 0 0 0

temp2: 0 0 0 0 1 1 0 1

temp1 |= temp2: 1 1 0 1 1 1 0 1



K0	0xEE	K8	0xEB
K1	0xDE	K9	0xDB
K2	0xBE	K10	0xBB
K3	0x7E	K11	0x7B
K4	0xED	K12	0xE7
K5	0xDD	K13	0xD7
K6	0xBD	K14	0xB7
K7	0x7D	K15	0x77



• 标识符

- 用来标识变量名、符号常量名、函数名、类型名、文件名等的有效字符序列。
- 规则：
 - 标识符只能由**字母、数字和下划线**三种字符组成，且第一个字符必须为字母或下划线。
 - 用户自定义的标识符不能与**关键字**同名。

• 关键字

- 关键字 (key word) 是一种具有固定名称和特定含义的标识符
- ANSI C语言定义的关键字有32个
- **Keil C51扩展关键字**



存储器类型相关

<u>_at_</u>	为变量进行存储器绝对空间定位
bit	声明一个位变量或者位类型的函数
code	8051单片机程序存储器
bdata	可位寻址的8051单片机内部数据存储器
data	直接寻址的8051单片机内部数据存储器
idata	间接寻址的8051单片机内部数据存储器
pdata	“分页”寻址的8051单片机外部数据存储器
xdata	8051单片机的外部（扩展）数据存储器
far	扩充的RAM和ROM（超出64K）



特殊功能寄存器相关



sbit	定义一个可位寻址变量地址
sfr	定义一个8位特殊功能寄存器地址
sfr16	定义一个16位特殊功能寄存器地址

存储模式相关

compat	指定使用8051外部分页寻址数据存储空间
large	指定使用8051外部数据存储空间
small	指定使用8051内部数据存储空间



函数相关

interrupt	定义一个中断函数
reentrant	定义一个再入函数
alien	用于声明与PL/M51兼容函数

其他

using	函数内部使用的8051的工作寄存器组
task	定义实时多任务函数
priority	规定RTX51或RTX51 Tiny的任务优先级



第一部分

1. 存储器结构
2. 中断
3. I/O
4. 复位与时钟
5. 按键与显示

第二部分

1. 定时器
2. 串口
3. AD/DA



怎么样写一个规范完整的程序?

```
#include <c8051f020.h> //头文件
sbit P1_0 = P1^0; //定义全局变量
char data *p;
int num = 0;
```

```
//定义子函数
Oscillator_Init()
{
    .....
}
Watchdog_Init()
{
    .....
}
Timer0_Init()
{
    .....
}
UART0_Init()
{
    .....
}
ADC0_Init()
{
    .....
}
```

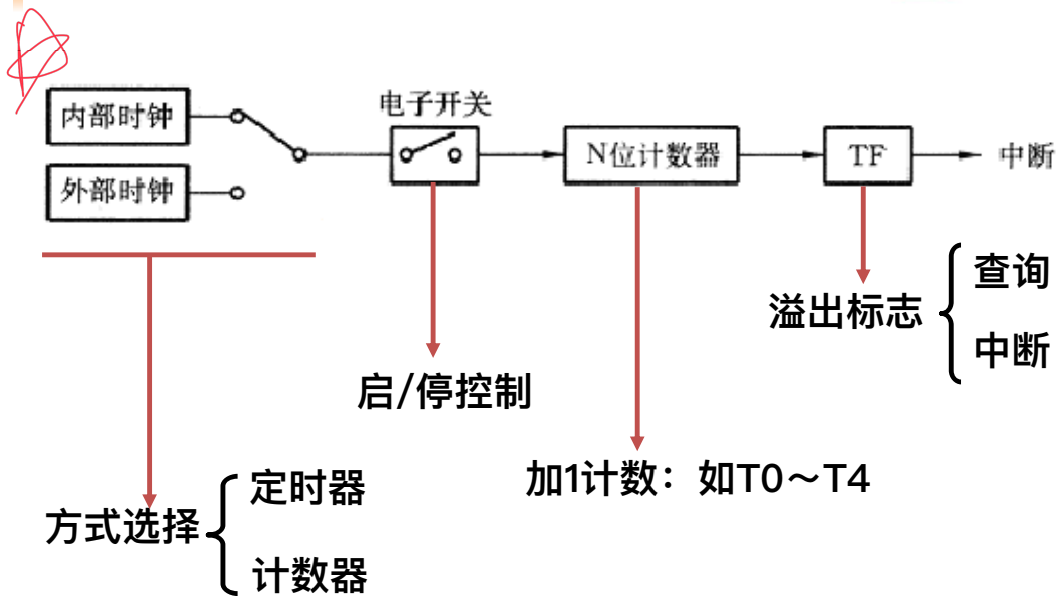
```
//主函数
void main()
{
    Watchdog_Init();
    Oscillator_Init();
    Timer0_Init();
    UART0_Init();
    ADC0_Init();
    while(1); //死循环, 等待中断
}

//中断服务程序
void Timer0_ISR(void) interrupt 1
{
    .....
}
void interrupt_UART0(void) interrupt 4
{
    .....
}
void ADC0_ISR (void) interrupt 15
{
    .....
}
```

代码有注释, 会画流程图!

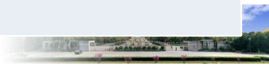


4.2 定时器的一般结构和工作原理



定时器/计数器工作方式

	T0/T1	T2/T4	T3
方式0	13位定时器/计数器	自动重载的16位定时器/计数器	自动重载的16位定时器/计数器
方式1	16位定时器/计数器	带捕捉的16位定时器/计数器	
方式2	8位自动重载的定时器/计数器 (波特率发生器)	UART0/UART1的波特率发生器	
方式3	两个8位定时器/计数器 (只限于定时器0)		



1. 选用哪个定时器? T0/T1的最大定时时间 ($f_{OSC}=12\text{MHz}$ 、 $T0M=0$) :

2.用哪个工作方式?

方式0: $T_{MAX} = 2^{13} \times 1\mu s = 8.192\text{ms}$

方式1: $T_{MAX} = 2^{16} \times 1\mu s = 65.536\text{ms}$

方式2、3: $T_{MAX} = 2^8 \times 1\mu s = 0.256\text{ms}$

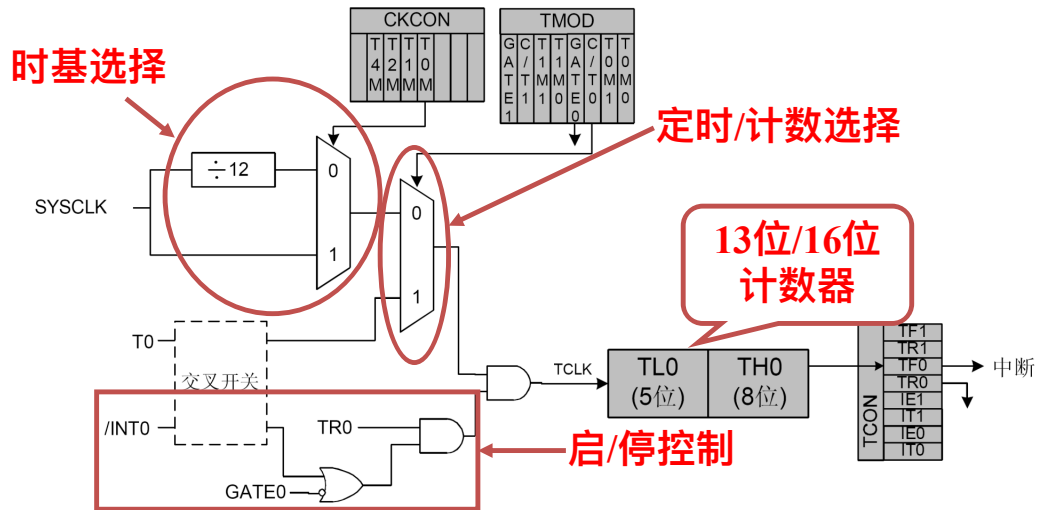
•初始化步骤

- 初始化TMOD (选择工作方式)
- 根据需要初始化CKCON (是否12分频, 另OSCXCN控制是否晶振2分频)
- 装入初值 (计算初值 $a = 2^N - \frac{T}{12^{1-T0M}} \times f_{osc}$), 赋值给 TH0/TH1、TL0/TL1
- 中断设置 (IE、IP)
- 启动定时/计数器 (TCON)



定时器T0和T1

工作方式0/1



GATE0=0 TR0= ———→ 启动计数

时:

1

GATE0=1 TR0=1且 ———→ 启动计数

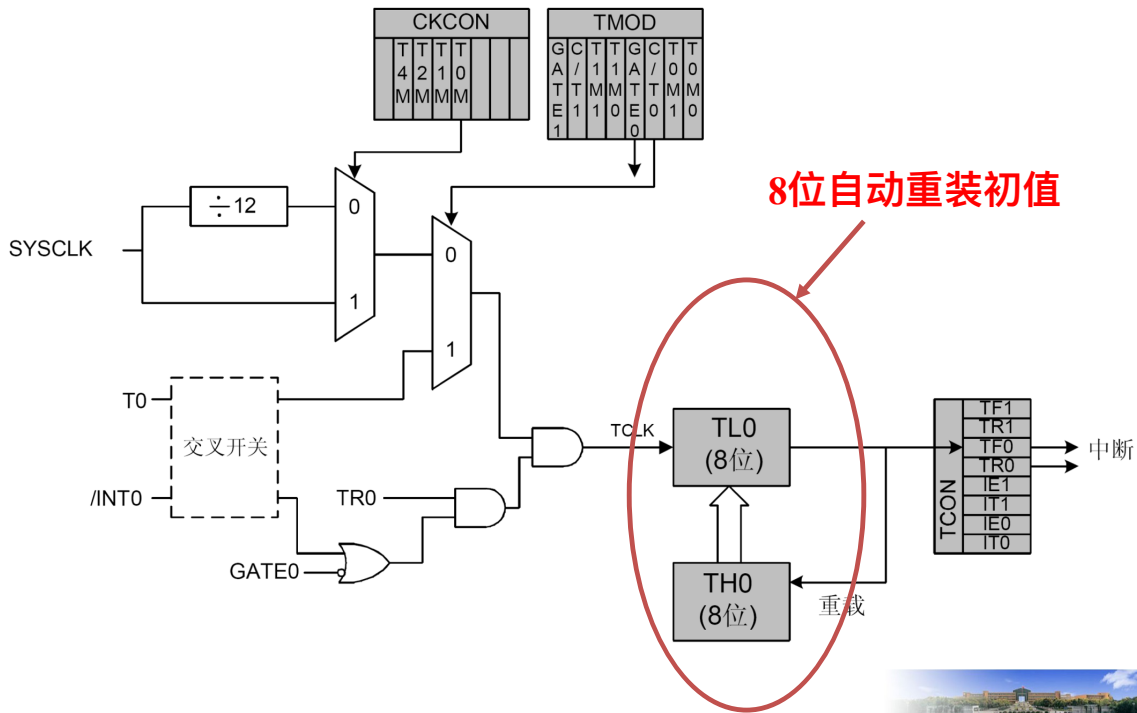
时:

INT0=1



定时器T0和T1

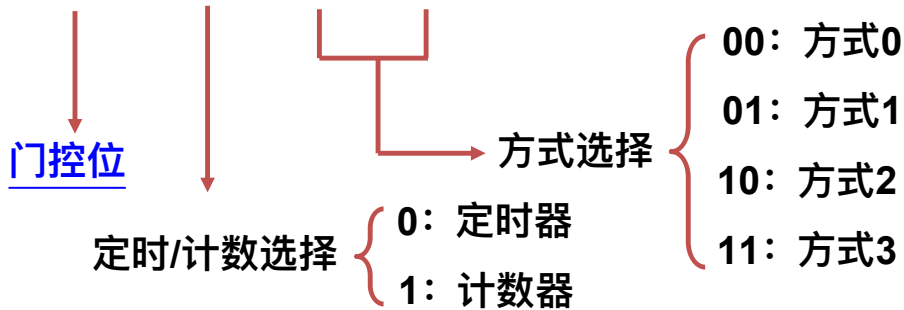
工作方式2



初始化TMOD TMOD=0x01; //T0, 方式1

位7	位6	位5	位4	位3	位2	位1	位0	复位值
GATE1	C/ $\overline{T1}$	T1M1	T1M0	GATE	C/ $\overline{T0}$	T0M1	T0M0	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0x89

0 0 0 0 0 0 0 1



► 初始化CKCON

CKCON &= 0xF7; //12分频

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	T4M	T2M	T1M	T0M	保留	保留	保留	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0x8E

0

•位6-3: T4M-T0M: T4到T0的时钟选择 (不含T3, T3的时钟选择由T3控制寄存器TMR3CN的第0位T3XCLK决定)。

0: 定时器按系统时钟的12分频计数

1: 定时器按系统时钟频率计数



► 中断设置 (IE、IP) 与寄存器地址

• IE $IE = 0x02;$ //允许T0中断

位7	位6	位5	位4	位3	位2	位1	位0	复位值
EA	IEGF0	ET2	ES0	ET1	EX1	ET0	EX0	00000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xA8

• IP $IP = 0x02;$ //T0中断为高优先级

位7	位6	位5	位4	位3	位2	位1	位0	复位值
-	-	PT2	PS0	PT1	PX1	PT0	PX0	11000000
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	SFR地址: 0xB8



工作方式0

[illegible]

6. T0和T1的应用举例

• **例4.2** 若 $f_{OSC}=12\text{MHz}$, T1工作于方式1, 产生50ms的定时中断, TF1为高级中断源。试编写主程序和中断服务程序, 使P1.0产生**周期为1s**的方波。

• **解:** 让P1.0每**500ms**取反一次即可实现。定时器的单次定时时间不可能达到500ms, 可让定时器**多次定时产生500ms**的定时时间, 如让T1工作在方式1, 单次定时时间为50ms, 那么T1中断10次就是500ms的时间。

• (1) 确定定时常数

– 假设使用 f_{OSC} 的12分频作为计数源, 则 $T_{\text{计数}} = 12 / f_{OSC} = 12 / (12 \times 10^6) = 1\mu\text{s}$

– 由公式 $TC = M - T / T_{\text{计数}}$, 可知 $TC = 2^{16} - 50 \times 10^3 = 15536 = 3CB0H$



程序清单如下（主程序）：

```
#include <c8051f020.h>

sbit P1_0 = P1^0;

int count=10;           //10次T1中断为500ms

void main( void )
{
    Watchdog_Init();
    Oscillator_Init();
    Timer1_Init();
    P1_0=0;
    while(1);           //死循环，等待中断，产生方波
}
```



 void Timer1_Init()

{

CKCON&=0xef; //T1的计数源选择系统脉冲的12分频

TMOD=0x10; //T1方式1

TH1=0x3c; //初值

TL1=0xb0;

IE|=0x88; //允许T1中断

IP|=0x08; //TF1中断为高级中断

TCON|=0x20; //启动T1

}



```
void Oscillator_Init()
{
    int i = 0;
    OSCXCN    = 0x77;
    for (i = 0; i < 3000; i++);
    while ((OSCXCN & 0x80) == 0);
    OSCICN    = 0x08;
}

void Watchdog_Init()
{
    EA = 0;
    WDTCN=0XDE;
    WDTCN=0XAD;
    EA = 1;
}
```



程序清单如下（中断服务程序）：

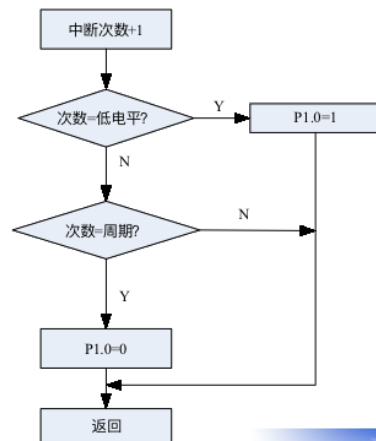
```
void Timer1_ISR (void) interrupt 3
{
    TH1=0x3c;           //重装初值
    TL1|=0xb0;
    count--;            //中断计数
    if (count==0)       //500ms到，重赋计数初值，P1.0取反
    {
        count=10;  P1_0=!P1_0;
    }
}
```



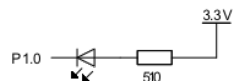
1. 使用定时器0，用中断方法设计一个秒闪电路，让发光二极管LED每秒钟有400ms点亮。假定C8051F020单片机定时器0的时钟频率为1MHz，发光二极管接在P1.0口；要求：

- (1) 设计相应的接口电路。
- (2) 画出中断程序流程图。
- (3) 编写定时器中断程序，要有必要的注释。（本题20分）

(2) 中断程序流程图



(1) 设计相应的接口电路



(3) 定时器中断程序

```
#include <c8051f020.h>
#define uchar unsigned char
sbit WAVE=P1^0;
uchar    time;
uchar    PERIOD = 1000;
uchar    LOW= 400;

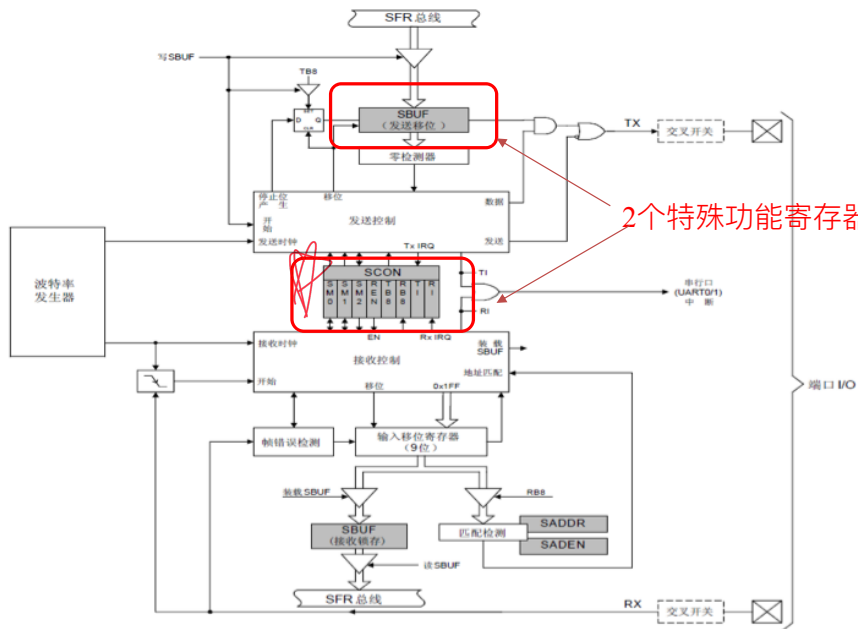
Timer0_ISR () interrupt 1    //定时器每1ms溢出
{
    TH0 = ( 65536 - 1000) / 256;    //TH0,TL0重装初值
    TL0 = ( 65536 - 1000 ) % 256;
    time = time + 1;
    if( time == LOW) WAVE=1; //高电平时间到了变高电平
    else if ( time == PERIOD)
    {
        time=0;
        WAVE = 0; //周期到了变低电平
    }
}
```



 宁波大学
NINGBO UNIVERSITY

UART0以T1或T2作为波特率发生器，UART1以T1或T4作为波特率发生器

2个特殊功能寄存器



UART0的4种工作方式

SM00	SM10	方式	功 能 说 明
0	0	0	同步方式（扩展移位寄存器方式，用于I/O口扩展），波特率固定（ $f_{osc} / 12$ ）
0	1	1	8位UART，波特率可变（由T1或T2溢出率决定）
1	0	2	9位UART，波特率固定（ $f_{osc} / 64$ 或 $f_{osc} / 32$ ）
1	1	3	9位UART，波特率可变（由T1或T2溢出率决定）



初始化步骤

1. 设定SCON

- (1) 设定工作方式SM00、SM10
- (2) 需要接受数据, REN=1 (允许接收)
- (3) 将TI、RI进行清零

2. 设定IE

是否允许中断, EA = 1; ES0=1

3. 交叉开关配置 (是否需要输出)

XBR0 = 0X04; XBR2 = 0x40; 将TXn和RXn配置给P0.0和P0.1

4. 定时器T1/T2初始化 (计算波特率)

按照定时器初始化步骤, TMOD、CKCON、IE、IP、TCON

PCON的最高位为SMOD0, 决定波特率是否加倍



根据波特率计算定时器初值

- 方式0: $f_{OSC} / 12$
- 方式2: $2^{SMOD} \times f_{OSC} / 64$

- 方式1、3:

用定时器T1的方式2产生波特率

波特率是否加倍?

时钟是否12分频?

$$X = 256 - \frac{2^{SMOD0} \times SYSCLK \times (12^{(T1M-1)})}{32 \times \text{波特率}}$$

定时器T2和T4产生波特率

方式 1 和方式 3 波特率 =
$$\frac{SYSCLK}{32 \times (65536 - [RCAP2H: RCAP2L])}$$



- ▶ 采用**查询和中断**两种方式。TI和RI是一帧数据是否发送完或收齐的标志，可用于CPU查询；如果允许，也可以向CPU申请中断。两种方式的编程方法如下：
 - **查询式发送程序**：发送一个数据→查询TI→发送下一个数据（先发后查）。
 - **查询式接收程序**：查询RI→读入一个数据→查询RI→读下一个数据（先查后收）。
 - **中断式发送程序**：发送一个数据→等待中断→在中断服务程序中发送下一个数据。
 - **中断式接收程序**：等待中断→在中断服务程序中接收一个数据。
- ▶ 两种方式中，发送和接收数据后都需要注意**清零TI或RI**



3. 串行通信编程举例

•**例4** 在C8051F020的片内数据存储器20H~3FH单元共有32个字节数据，要求使用UART0方式1发送出去，传输波特率为9600，设SYSCLK=12MHz。试分别用查询方式和中断方式编写发送程序和相应的接收程序。

•**解** 选T1工作于方式2，作为波特率发生器，取T1M=0（T1按振荡器的12分频计数）、SMOD0=0，T1的时间常数计算
253 (FDH)

$$\therefore X = 256 - \frac{2^{SMOD0} \times SYSCLK \times (12^{(T1M-1)})}{32 \times \text{波特率}}$$



查询方式发送程序:

```
#include <c8051f020.h>
void main(void)
{
    unsigned char i;
    char data *p;
    XBR0=0x04;           //发送数据块地址指针
    XBR2=0x40;           //配置交叉开关, 使能UART0
    P0MOUT|=0x01;        //使能交叉开关
    TMOD=0x20;           //TX0为推挽输出
    TH1=0xFD;            //初始化并启动T1, 方式2
    TL1=0xFD;
    TR1=1;
    SCON0=0x40;          //UART0初始化, 方式1
    p=0x20;              //地址指针初始化
    for(i=0;i<32;i++)
    {
        SBUF0=*p;        //一字节送发送SBUF0
        p++;
        while(!TI0);     //等待发送完成
        TI0=0;
    }
}
```



查询方式接收程序:

```
#include <c8051f020.h>
void main(void)
{
    unsigned char i;
    char data *p;                //发送数据块地址指针
    XBR0=0x04;
    XBR2=0x40;
    P0MOUT|=0x01;
    TMOD=0x20;                   //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x50;                  //UART0初始化, 方式1、允许接收
    p=0x20;                      //地址指针初始化
    for(i=0;i<32;i++)
    {
        while(!RI0);            //等待UART0接收一个字符
        RI=0;
        *p=SBUF0;               //放入接收缓冲区
        p++;
    }
}
```



中断方式发送主程序:

```
#include <c8051f020.h>
char data *p;           //发送数据块地址指针
void main(void)
{
    XBR0=0x04;           //配置交叉开关
    XBR2=0x40;
    P0MOUT|=0x01;
    TMOD=0x20;           //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x40;           //UART0初始化, 方式1
    p=0x20;               //地址指针初始化
    EA=1;                 //开中断
    ES0=1;
    SBUF0=*p;             //发送第一个字符
    while(1);             //等待发送中断
}
```



中断方式发送中断服务程序:

```
void interrupt_UART0(void) interrupt 4
{
    TI0=0;           //清发送中断标志
    p++;
    if(p<0x40)
        SBUF0=*P;    //发送下一字节
    else
        ES0=0;       //关串口中断
}
```



中断方式接收主程序:

```
#include <c8051f020.h>
char data *p;           //发送数据块地址指针
void main(void)
{
    unsigned char i;
    XBR0=0x04;           //配置交叉开关
    XBR2=0x40;
    P0MOUT|=0x01;
    TMOD=0x20;           //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x50;           //UART0初始化, 方式1、允许接收
    p=0x20;               //地址指针初始化
    EA=1;                 //开串口中断
    ES0=1;
    while(1);             //等待接收中断
}
```



中断方式接收中断服务程序:

```
void interrupt_UART0(void) interrupt 4
{
    RI0=0;           //清接收中断标志
    *p=SBUF0;         //收到的字符送接收缓冲区
    p++;
    if(p>=0x40) ES0=0; //关串口中断
}
```



3. 串行通信编程举例

• **例5** 试编写一个UART0带奇偶校验的发送程序。

• 设SYSCLK=11.0592MHz,波特率=9600, UART0工作于方式1,
发送字符的**ASCII码最高位作校验位**, 用T2作波特率发生器,
T2的时间常数计算如下:

$$\diamond \therefore \text{波特率} = \frac{SYSCLK}{32 \times (65536 - [RCAP2H: RCAP2L])}$$

$$\diamond \therefore RCAP2H: RCAP2L = 65536 - \frac{SYSCLK}{32 \times \text{波特率}}$$

$$= 65500 = 0\text{xffdc}$$



```
#include <c8051f020.h>
#include <string.h>
char s[]="C8051F020 Serial Communication";
char bdata c;      //可位寻址的内部数据存储区
sbit c7=c^7;
void main(void)
{
    char a,b=0;
    XBR0=0x04;      //配置交叉开关, 使能UART0
    XBR2=0x40;      //使能交叉开关
    P0MOUT|=0x01;   ///TX0为推挽输出(错), 总是漏极开路
    T2CON=0x14;     //T2作发送波特率发生器,TCLK0=1,TR2=1
    SCON0=0x40;     //SM20=SM00=0,SM10=1,REN0=0;
    RCAP2H=0xff;
    RCAP2L=0xdc;
    a=strlen(s);
```



```
for(;b<a;b++)
```

```
{
```

```
    c=s[b];
```

```
    ACC=c;
```

//将字符送到累加器ACC中进行校验，形成校验位P

```
    c7=P;
```

//最高位作校验位，P即PSW.0；ASCII码最高位恒为0

```
    SBUF0=c;
```

```
    while(!TI0);
```

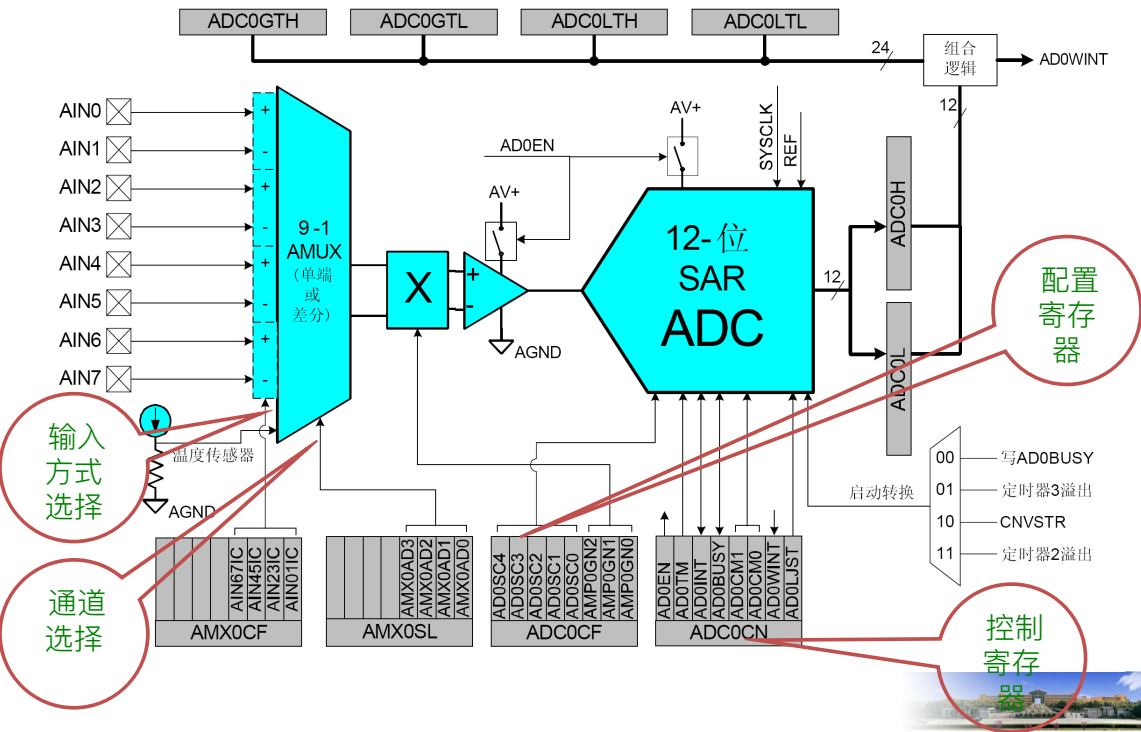
```
    TI0=0;
```

```
}
```

```
}
```



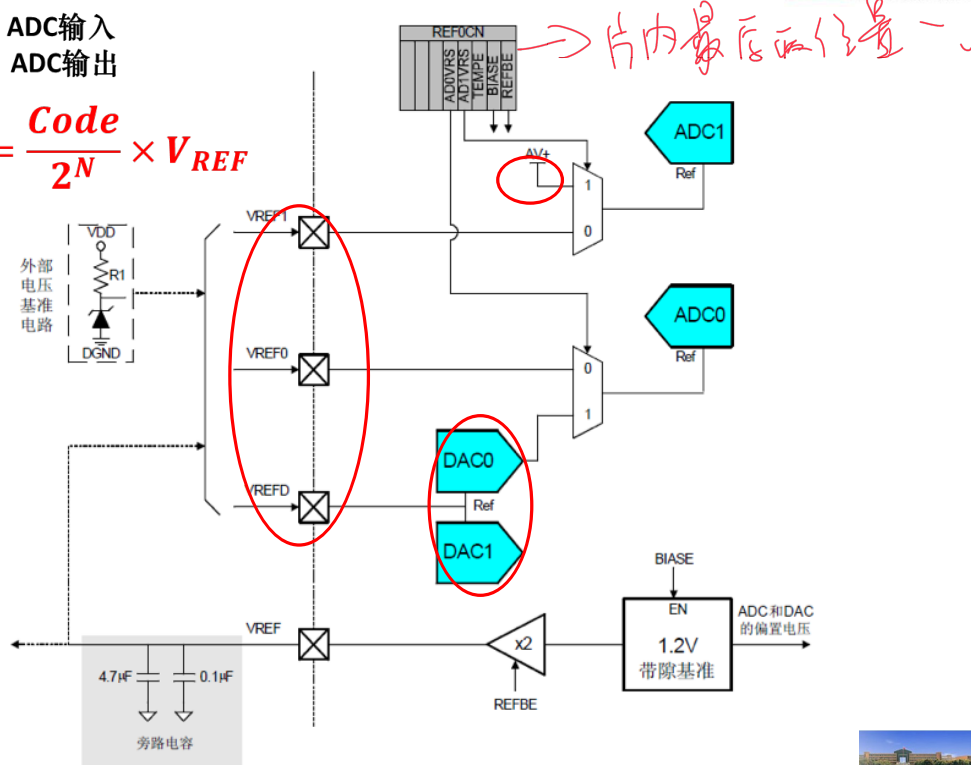
ADC0子系统功能框图



V_{in} : ADC输入

Code: ADC输出

$$V_{in} = \frac{\text{Code}}{2^N} \times V_{REF}$$



• ADC的初始化:

- 配置**ADC0CN**设置ADC0使能、启动方式和数据对齐方式
- 配置**AMUX**配置寄存器 (**AMX0CF**) 设置模拟量测试为单端或差分输入
- 配置**AMX0SL**选通道
- 确定ADC配置寄存器 (**ADC0CF**) 设定ADC转换速度和对模拟量的放大倍数;
- 确定基准电压, 配置基准寄存器 (**REF0CN**);

• ADC的使用步骤 (查询方式):

- 清零转换完成标志位 (**AD0INT = 0**)
- 启动转换 (**ADC0BUSY=1**)
- 查询是否转换结束 (**while (AD0INT ==0)**)
- 等转换结束后把数据转移出来 (**temperature = ADC0**)



sfr16 ADC0 = 0xBE; //ADC0数据, ADC0L和ADC0H串起来

long temperature;

main (void)

{

ADC0_Init (); // 初始化和使能ADC

while (1)

{

AD0INT = 0; //清除转换结束标记

AD0BUSY = 1; // 开始转换

while (AD0INT == 0); // 等待转换结束

temperature = ADC0; // 读ADC0数据

数据处理.....

}

}



ADC0初始化

```
void ADC0_Init (void)
```

```
{
```

```
    ADC0CN = 0x81;    // ADC0使能
```

```
                        // 当写AD0BUSY时ADC0转换开始
```

```
                        //ADC0数据左对齐
```

```
    REF0CN = 0x07;    // 使能温度传感器片内VREF和VREF输出缓冲器
```

```
    AMX0SL = 0x0f;    // 选择温度传感器作为ADC多路模拟转换器输出
```

```
    ADC0CF = (SYSCLK/2500000) << 3;    // ADC转换时钟=2.5MHz
```

```
    ADC0CF |= 0x01;    // PGA增益=2
```

```
}
```



- ADC的初始化：
 - 配置ADC0CN设置ADC0使能、启动方式和数据对齐方式
 - 配置AMUX配置寄存器 (AMX0CF) 设置模拟量测试为单端或差分输入
 - 配置AMX0SL选通道
 - 确定ADC配置寄存器 (ADC0CF) 设定ADC转换速度和对模拟量的放大倍数；
 - 确定基准电压，配置基准寄存器 (REF0CN) ；
 - 开中断 (EIE2 |= 0x02 , EA=1)
- ADC的使用步骤 (中断方式) ：
 - 启动转换 (ADC0BUSY=1)
 - 中断服务程序ADC0_ISR中：
 - 清零转换完成标志位 (AD0INT = 0)
 - 数据转移出来 (temperature = ADC0)



```
sfr16 ADC0 = 0xBE; //ADC0数据，ADC0L和ADC0H串起来  
long temperature;
```

```
main (void)  
{  
    ADC0_Init ();           // 初始化和使能ADC  
    AD0BUSY = 1;           // 开始转换  
    while (1) ;  
}
```



ADC0初始化

```
void ADC0_Init (void)
```

```
{
```

```
    ADC0CN = 0x81;    // ADC0使能
```

```
                        // 当写AD0BUSY时ADC0转换开始
```

```
                        //ADC0数据左对齐
```

```
    REF0CN = 0x07;    // 使能温度传感器片内VREF和VREF输出缓冲器
```

```
    AMX0SL = 0x0f;    // 选择温度传感器作为ADC多路模拟转换器输出
```

```
    ADC0CF = (SYSCLK/2500000) << 3;    // ADC转换时钟=2.5MHz
```

```
    ADC0CF |= 0x01;    // PGA增益=2
```

```
    EIE2 |= 0x02;    // 允许ADC中断，如果有用到定时器溢出
```

```
    EA = 1;
```

```
}
```



中断服务程序

```
void ADC0_ISR (void) interrupt 15
```

```
{
```

```
    AD0INT = 0;                // 清除ADC转换结束标
```

```
    temperature = ADC0;        // 读ADC0数据
```

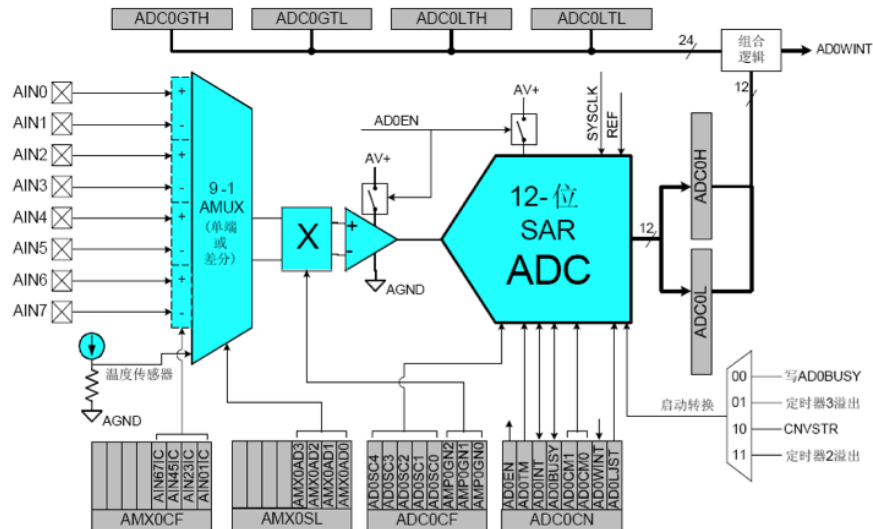
```
    数据处理.....
```

```
}
```

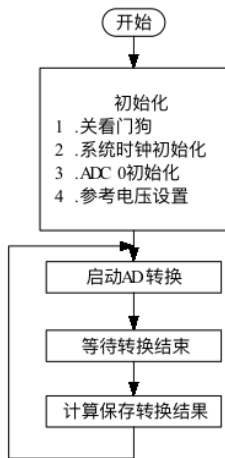


2. 已知C8051F020单片机12位ADC系统结构如图所示，试用ADC0用模拟输入**通道0**，采样外部直流电压信号，电压值保存于变量**Vin**中，参考电压选用片内参考电压（设为2.4V）。(20分)

- (1) 画出AD采集程序流程图。
- (2) 编写完整AD采集程序。要有必要的注释。



(1) 程序流程图



(2) 完整的程序

```

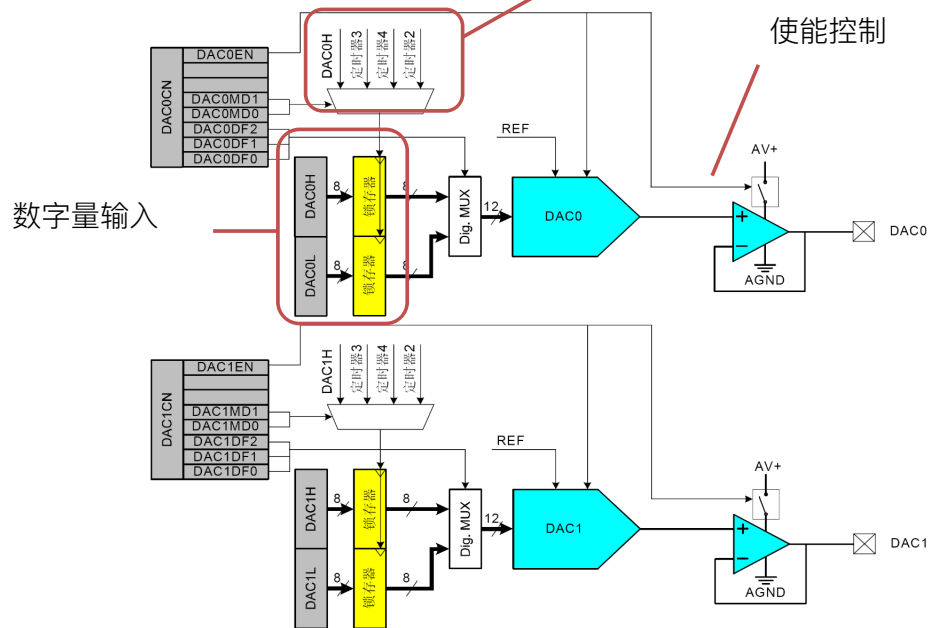
#include <c8051f020.h>
unsigned int ad_reslut;
float Vin;
void main( )
{
    WDTCN = 0xde;
    WDTCN = 0xad;      //关看门狗
    OSCXCN = 0x7f;     //设置外部晶振：二分频12MHz    Delay(1);
                    //延时等待外部晶振稳定
    OSCICN |= 0xf8;    //切换到外部晶振：1111_1000
    ADC0CF = 0x1f;     //ADC0转换时钟为系统时钟4分频，增益为1
    AD0EN = 1;         //ADC0使能
    REF0CN = 0x03;    //参考电压设置

    while(1)
    {
        AD0INT = 0;    //启动AD转换
        ADC0BUSY = 1;
        delay(1);
        while( AD0INT==0); //等待AD转换结束
        ad_reslut = ADC0H*256+ADC0L;
        Vin = 2.4* ad_reslut/4096;
    }
}
  
```



DAC 功能框图

输出更新方式



- DAC的初始化：→ 每138下米
 - 配置**DAC0CN**设置DAC0使能、启动方式和数据对齐方式
 - 确定基准电压，配置基准寄存器（**REF0CN**）；
 - DAC0L、DAC0H赋初值
- DAC的使用步骤：
 - 更新DAC的值（跟方式有关，写**DAC0H**或定时器溢出）



// DAC0用程序更新输出,产生一个阶梯波形。

```
sfr16 DAC0 = 0xd2;
```

```
#define UP 0x010
```

```
#define T=1000
```

```
void main(void)
```

```
{
```

```
    int i;
```

```
    WATCHDOG_init(); //禁止看门狗
```

```
    SYSCLK_init(); //外部晶振初始化
```

```
    DAC0_init (); //DAC0初始化
```

```
    for(i=0;i<=4095;i+UP)    //形成阶梯波形
```

```
    {
```

```
        DAC0=i;                //送数字量到DAC0直接更新输出
```

```
        d1ms(T);                //软件延迟
```

```
    }
```

```
}
```



看门狗、时钟、延迟函数

```
void WATCHDOG_init(); // 禁止看门狗
{
    EA = 0;
    WDTCN = 0xDE;
    WDTCN = 0xAD;
    EA = 1;
}
void SYSCLK_init(void)           //外部晶振初始化
{
    OSCXCN=0x67;
    for (n=0; n<255;n++)
        while(!(OSCXCN & 0x80));
    OSCICN=0x88;
}
void d1ms(int count)            //软件延迟
{ int j;
  while(count--!=0)
  { for (j=0;j<100;j++); }
}
```



```
void DAC0_init (void)           //DAC0初始化
{
    REF0CN = 0x03;               // 内部偏压发生器工作
    DAC0CN = 0x80;               // 允许DAC0，程序直接更新输出
                                   // 数据右对齐
    DAC0L = 0x00;                // DAC0数据寄存器初值
    DAC0H = 0x00;
}
```



(2) 产生锯齿波

```
sfr16 T2 = 0xcc;
```

```
Sfr16 DAC1=0xd5;
```

```
void main(void)
```

```
{
```

```
    WATCHDOG_init(); //禁止看门狗
```

```
    SYSCLK_init();    //外部晶振初始化
```

```
    DAC1_init ();     //DAC1初始化
```

```
    Timer2_init();
```

```
    while(1);
```

```
}
```



看门狗、时钟

```
void WATCHDOG_init(); // 禁止看门狗
```

```
{
```

```
    EA = 0;
```

```
    WDTCN = 0xDE;
```

```
    WDTCN = 0xAD;
```

```
    EA = 1;
```

```
}
```

```
void SYSCLK_init(void)           //外部晶振初始化
```

```
{
```

```
    OSCXCN=0x67;
```

```
    for (n=0; n<255;n++)
```

```
        while(!(OSCXCN & 0x80));
```

```
    OSCICN=0x88;
```

```
}
```



```
void DAC0_init (void)           //DAC0初始化
{
    REF0CN = 0x03;               // 内部偏压发生器工作
    DAC1CN = 0x98;               // 允许DAC1, T2溢出中中断更新
                                  // 数据右对齐
    DAC1L = 0x00;                // DAC0数据寄存器初值
    DAC1H = 0x00;
}
```



```
void Timer2_init(void)
```

```
{  
    RCAP2H = 0x05;           // 重新装入的时间常数  
    RCAP2L = 0x00;  
    TH2 = 0x05;              // 初始值  
    TL2 = 0x00;  
    T2CON = 0x04;            // 启动T2  
    IE = 0x20;               // T2中断允许  
}
```



```
void T2_ISR() interrupt 5
```

```
{
```

```
    TF2=0;           //清中断标志
```

```
    DAC1++;          //因为是T2溢出更新DAC1输出,
```

```
                    //所以可以对SFR16操作,
```

```
                    //此时并不立即更新
```

```
    if(DAC1>=0x1000)
```

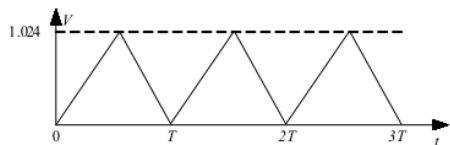
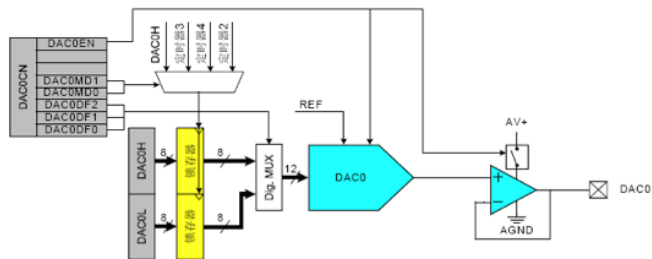
```
        DAC1=0;      //形成锯齿波
```

```
}
```

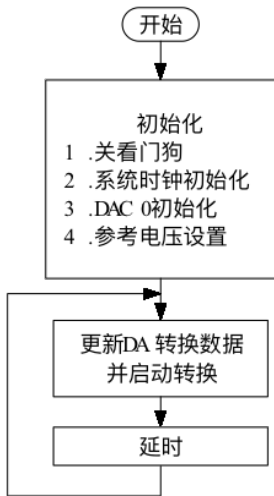


已知C8051F020单片机数模转换系统结构如图所示。试用DAC0设计一个三角波发生电路，波形如图所示，频率不限。要求参考电压选用片内参考电压（设为2.048V），要求：

- (1) 画出程序流程图。
- (2) 编写完整的程序。要有必要的注释。（本题20分）



(1) 程序流程图



(2) 完整的程序

```

#include <c8051f020.h>
unsigned int DA_TEMP, i;
sfr16 DAC0 = 0xd2; //DAC0为DAC0L与DAC0H的合并
unsigned int code tri_data[]={0,128,256, 384, 512,
640,768,896,1024,1152,1280,1480,1536,1664,1792, 1920,2048,1920, 1792,
1664,1536, 1480, 1280, 1152, 1024, 896, 768,640, 512,384, 256, 128,};
void main( )
{
    WATCHDOG_init(); //禁止看门狗
    SYSCLK_init(); //外部晶振初始化
    DAC0_init (); //DAC0初始化
    while(1)
    {
        DAC0 =tri_data[i];
        delay(1);
        i++;
    }
}

void DAC0_init (void) //DAC0初始化
{
    REF0CN = 0x03; // 内部偏压发生器工作
    DAC0CN = 0x80; // 允许DAC0, 程序直接更新输出, 数据
    DAC0 = 0x00; // DAC0数据寄存器初值
}
  
```

右对齐

